Software Testing, Verification, and Validation (M.EIC)
Software Testing (MESW)

Test Case Design

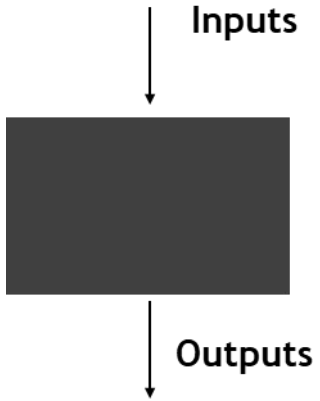
Ana Paiva

apaiva@fe.up.pt **www.fe.up.pt/~apaiva**

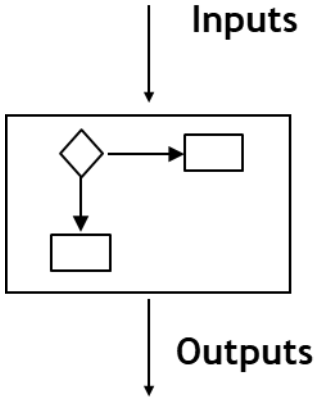
Test Design Strategies

Remember

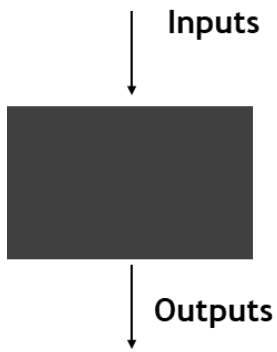
Black box (or Functional
or Close or …)



White box (or Structural
or Open or …)



Black box Testing



Remember

- **Knowledge sources**
  - Requirements document
  - Specifications
  - User manual
  - Models
  - Domain knowledge
  - Defect analysis data
  - Intuition
  - Experience

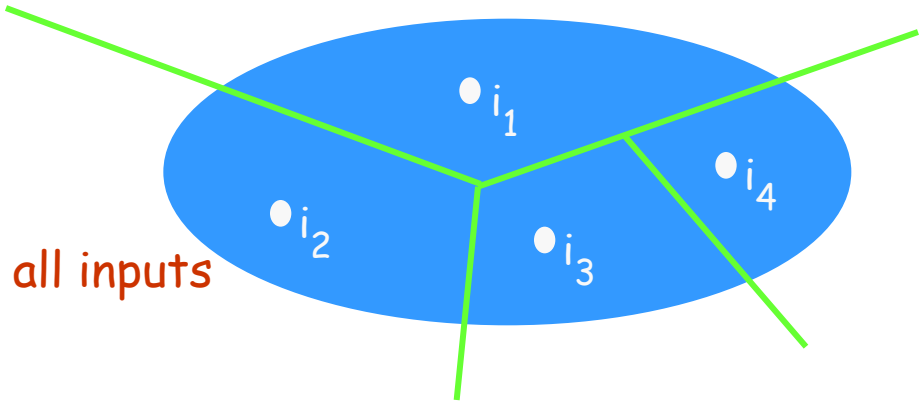
- **Techniques / Methods**
  - Equivalence class partitioning
  - Boundary value analysis
  - Cause effect graphing
  - Pairwise testing
  - Error guessing
  - Random testing
  - State-transition testing
  - Scenario-based testing

Equivalence class partitioning

Divide all possible inputs into classes (partitions) such that :

- There is a finite number of input equivalence classes
- You may reasonably assume that
  - The program behaves analogously for inputs in the same class
  - One test with a representative value from a class is sufficient
  - If the representative detects a defect, then other class members would detect the same defect

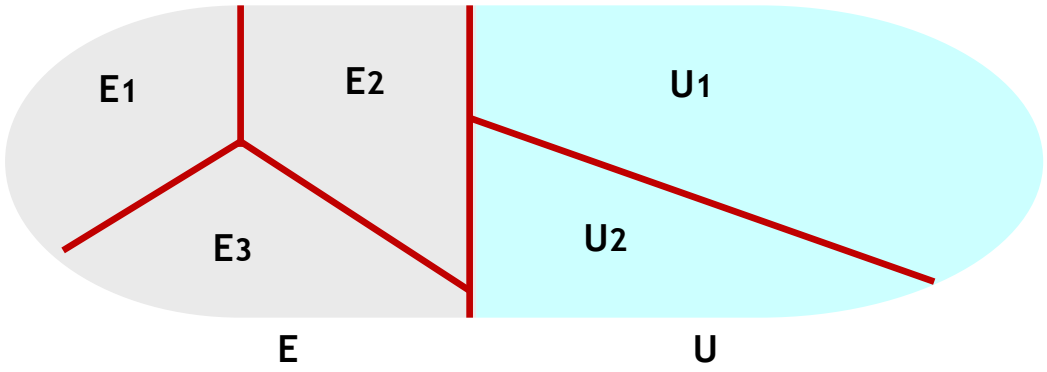
(Can also be applied to outputs)



Equivalence class partitioning

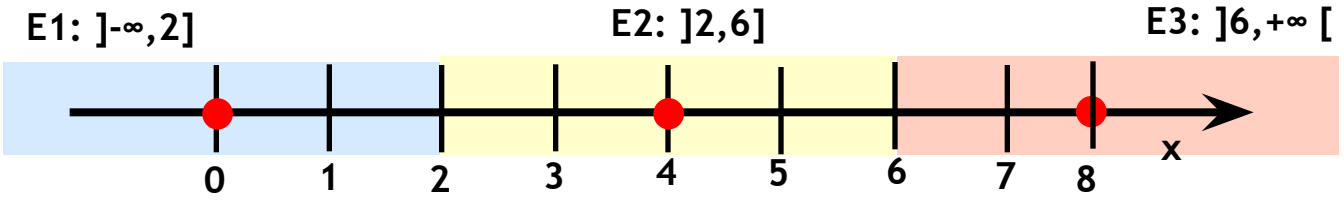
Faults targeted

- The entire set of inputs to any application can be divided into at least two subsets:
  - one containing all the expected, or legal, inputs (E) and
  - the other contains all unexpected or illegal inputs (U).
- Each of the two subsets can be further subdivided into subsets on which the application must behave differently (e.g., E1, E2, E3, and U1, U2).



Equivalence class partitioning

- Consider an application that
  - inputs one integer denoted by X
  - the only legal X values are in the range]2..6], i.e., X>2 and X<=6
- The input values are now divided into three subsets: integers in]2, 6], integers lower than or equal to 2, and integers higher than 6



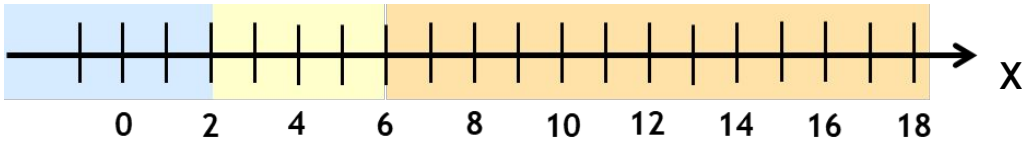
- Now we need 3 test cases, each with an input value from a different equivalence class, e.g., 0, 4, and 8

Equivalence class partitioning

- Now, consider an application that
  - inputs two integers denoted by X and Y
  - the only legal X values are in the range]2, 6], i.e., X>2 and X<=6
  - The only legal Y values are in the range [6, 9], i.e., Y>=6 and Y<=9

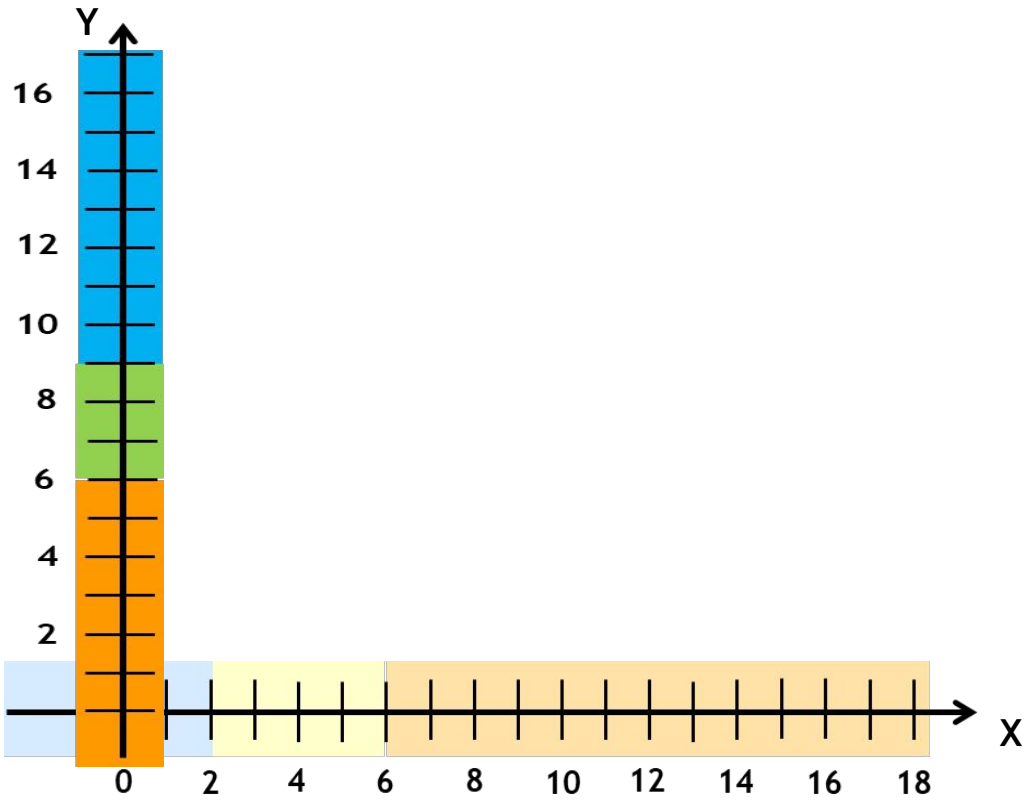
Equivalence class partitioning

- Now, consider an application that
  - inputs two integers denoted by X and Y
  - the only legal X values are in the range]2, 6], i.e., X>2 and X<=6
  - The only legal Y values are in the range [6, 9], i.e., Y>=6 and Y<=9



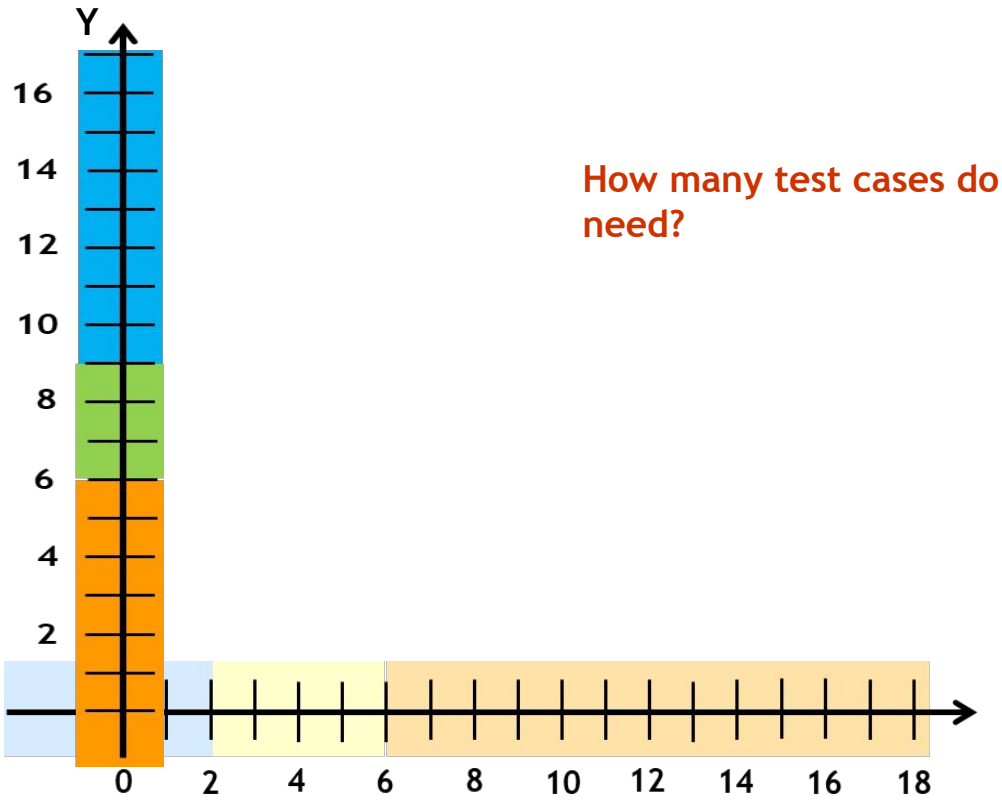
Equivalence class partitioning

- Now, consider an application that
  - inputs two integers denoted by X and Y
  - the only legal X values are in the range]2, 6], i.e., X>2 and X<=6
  - The only legal Y values are in the range [6, 9], i.e., Y>=6 and Y<=9



Equivalence class partitioning

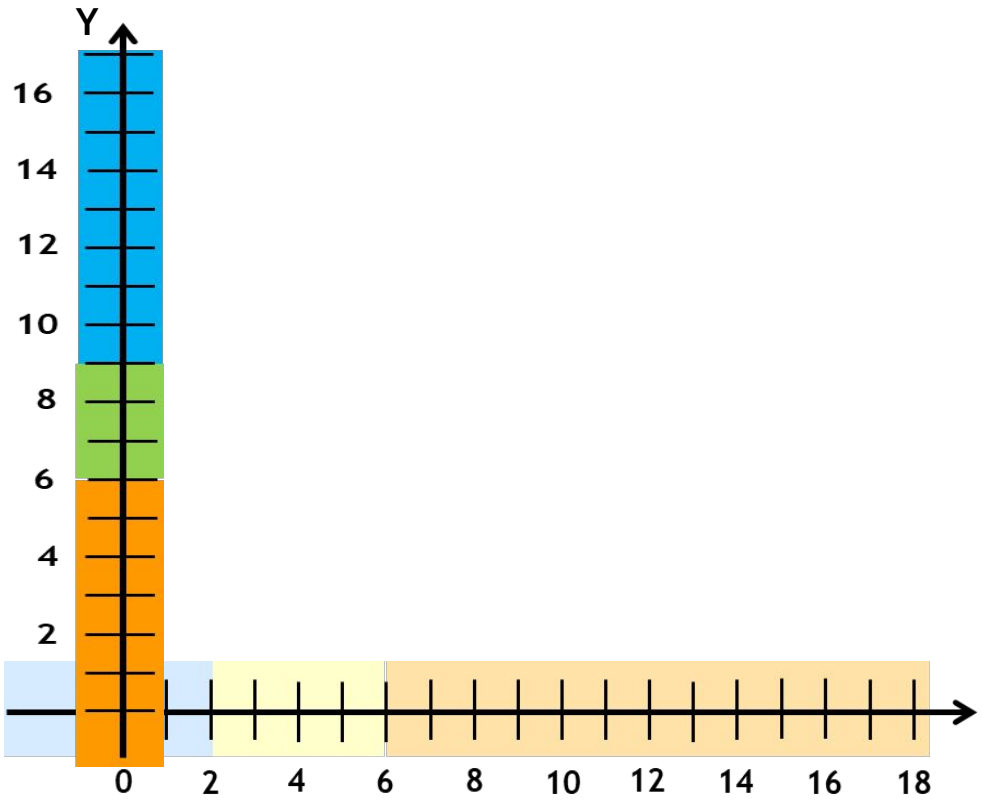
- Now, consider an application that
  - inputs two integers denoted by X and Y
  - the only legal X values are in the range]2, 6], i.e., X>2 and X<=6
  - The only legal Y values are in the range [6, 9], i.e., Y>=6 and Y<=9



How many test cases do we need?

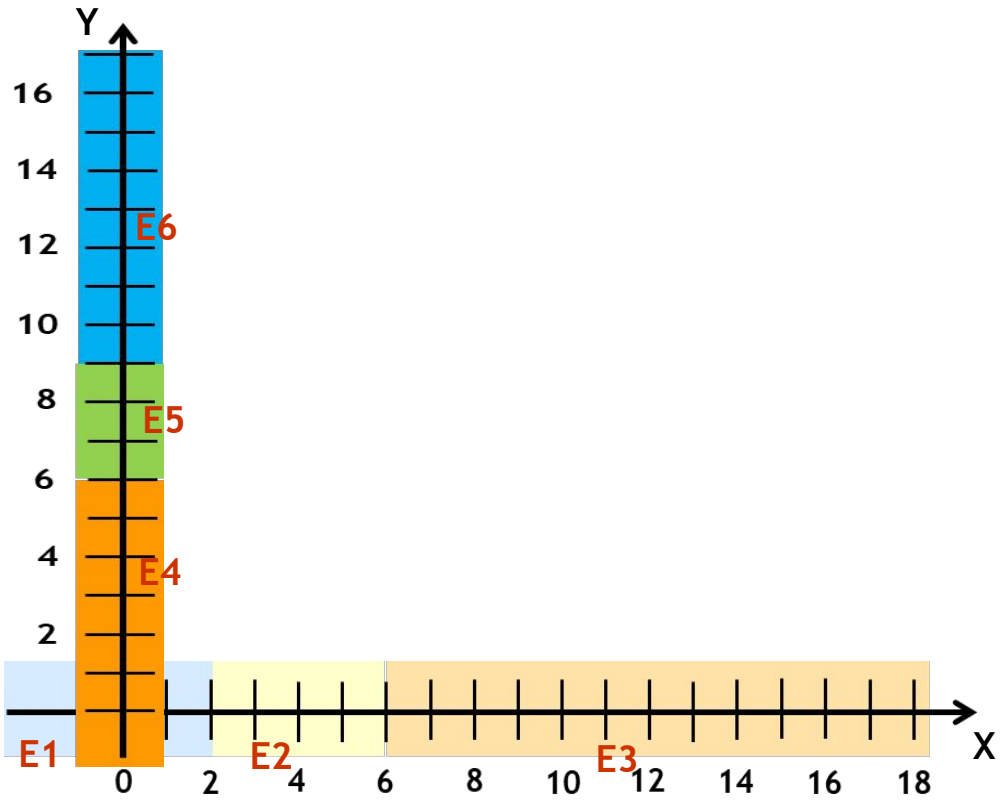
Equivalence class partitioning

- Unidimensional partitioning
  - Consider one input domain variable at a time.
  - Each input variable leads to a partition of the input domain.
  - We refer to this partitioning style as unidimensional equivalence or unidimensional partitioning.



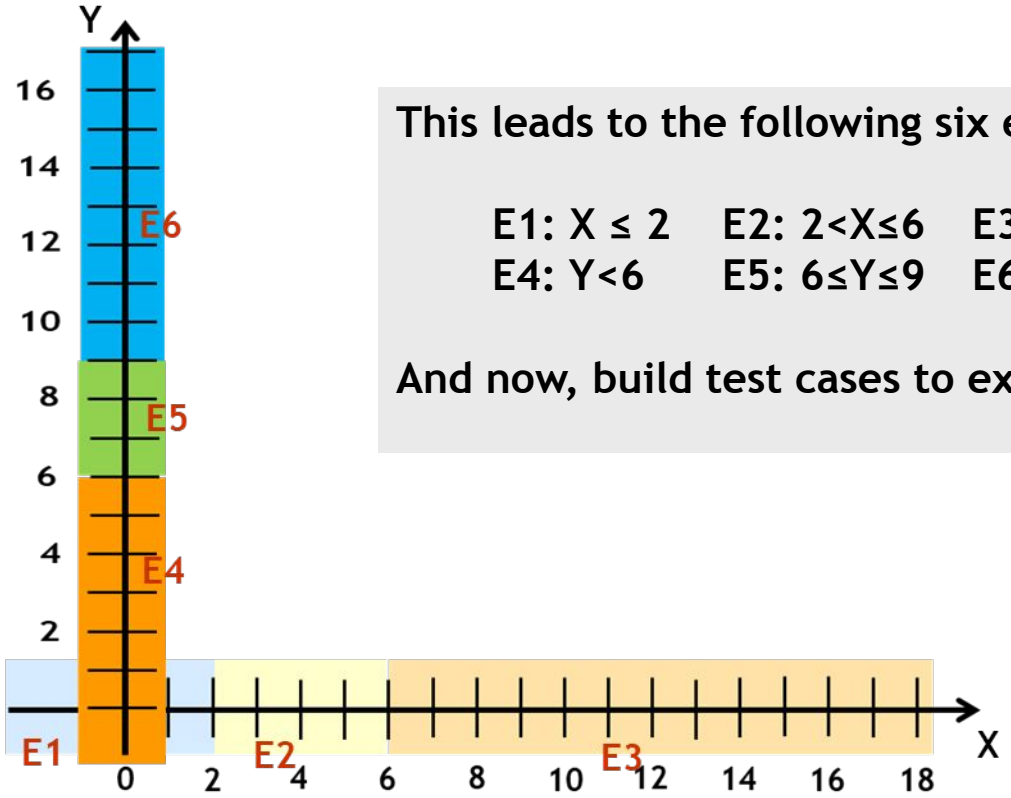
Equivalence class partitioning

- **Unidimensional partitioning**
  - Consider one input domain variable at a time.
  - Each input variable leads to a partition of the input domain.
  - We refer to this partitioning style as unidimensional equivalence or unidimensional partitioning.



Equivalence class partitioning

- **Unidimensional partitioning**
  - Consider one input domain variable at a time.
  - Each input variable leads to a partition of the input domain.
  - We refer to this partitioning style as unidimensional equivalence or unidimensional partitioning.



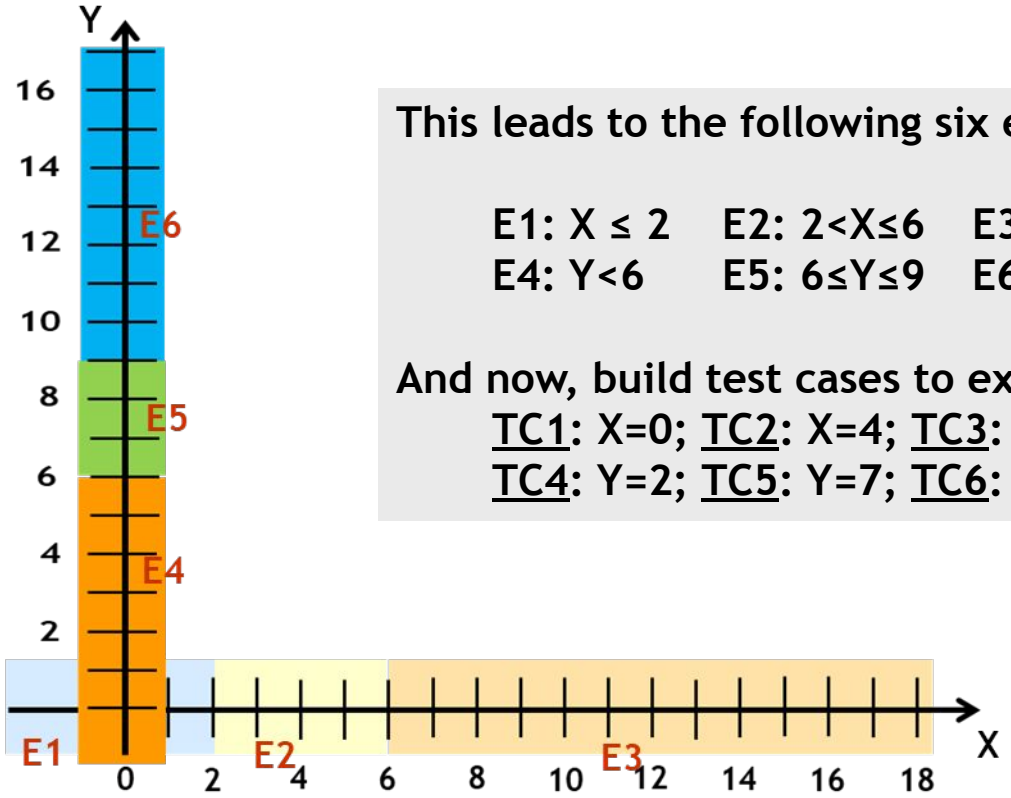
This leads to the following six equivalence classes:

E1: X ≤ 2 E2: 2<X≤6 E3: X>6 (Y ignored)
E4: Y<6 E5: 6≤Y≤9 E6: Y>9 (X ignored)

And now, build test cases to exercise those 6 classes

Equivalence class partitioning

- **Unidimensional partitioning**
  - Consider one input domain variable at a time.
  - Each input variable leads to a partition of the input domain.
  - We refer to this partitioning style as unidimensional equivalence or unidimensional partitioning.



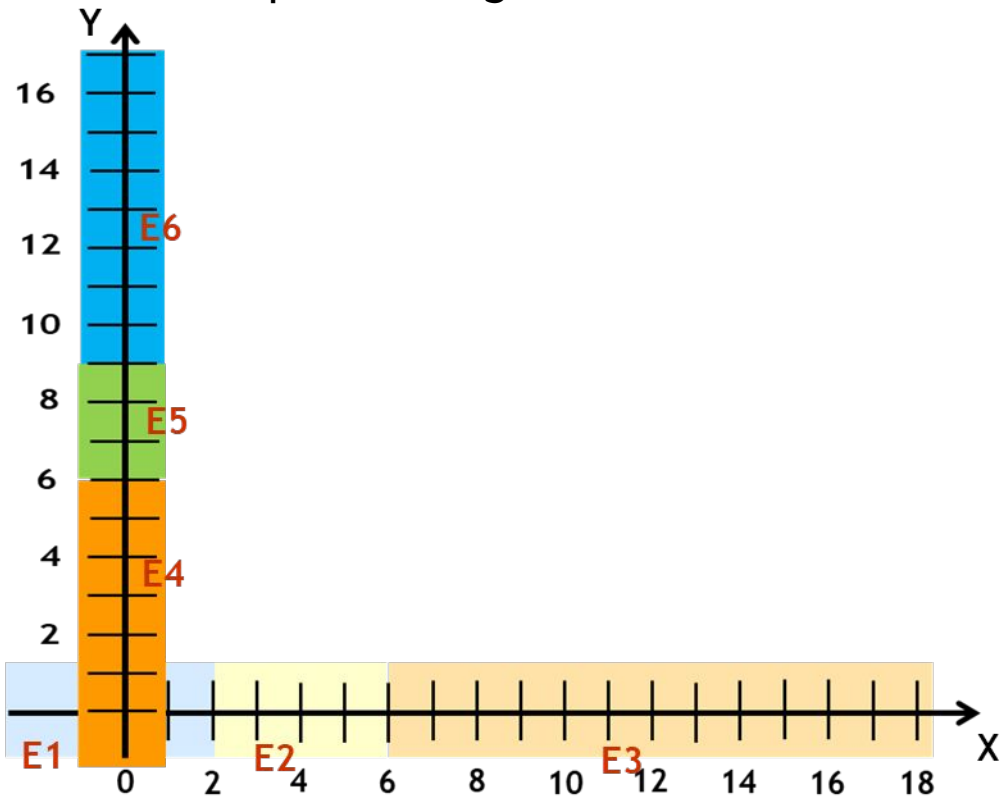
This leads to the following six equivalence classes:

E1: X ≤ 2 E2: 2<X≤6 E3: X>6 (Y ignored)
E4: Y<6 E5: 6≤Y≤9 E6: Y>9 (X ignored)

And now, build test cases to exercise those 6 classes
TC1: X=0; TC2: X=4; TC3: X=8
TC4: Y=2; TC5: Y=7; TC6: Y=10

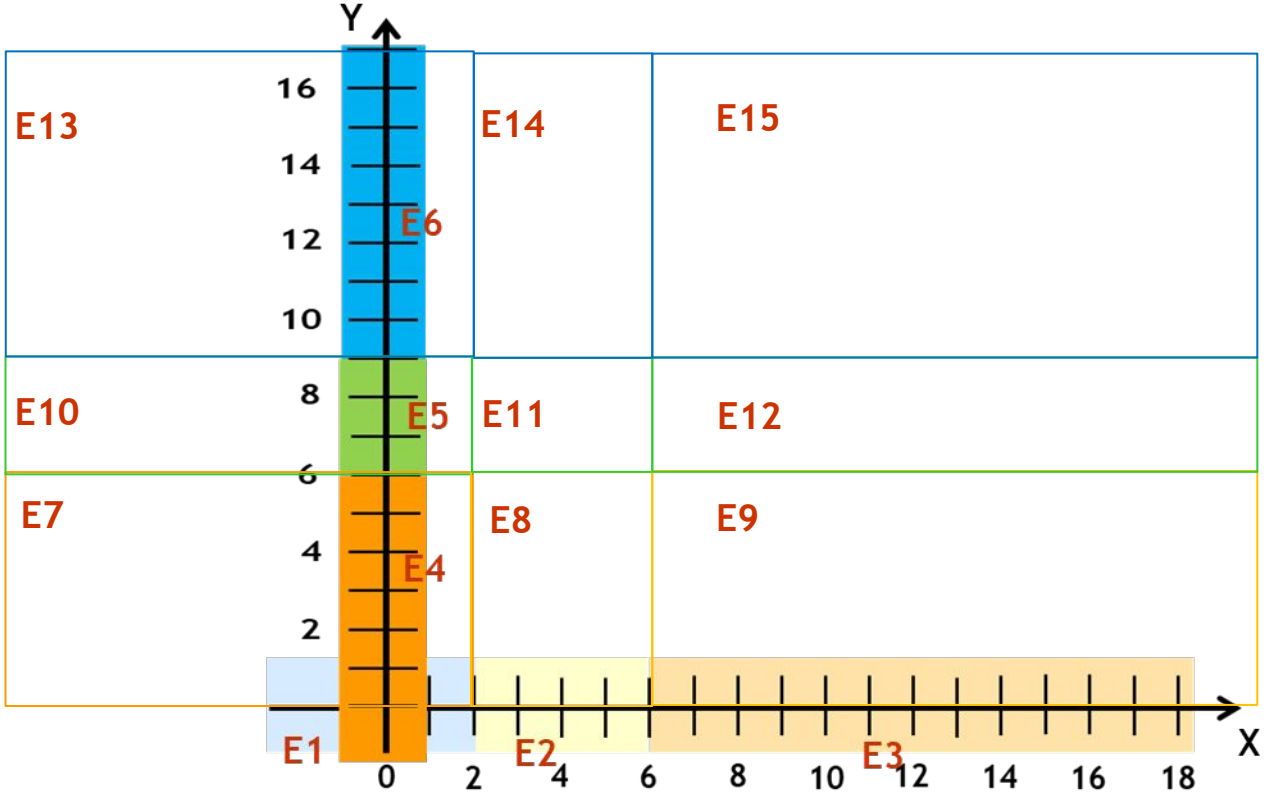
Equivalence class partitioning

- **Multidimensional partitioning**
  - Consider the input domain as the set product of the input variables.
  - One partition consisting of several equivalence classes.
  - We refer to this method as multidimensional equivalence partitioning or multidimensional partitioning.



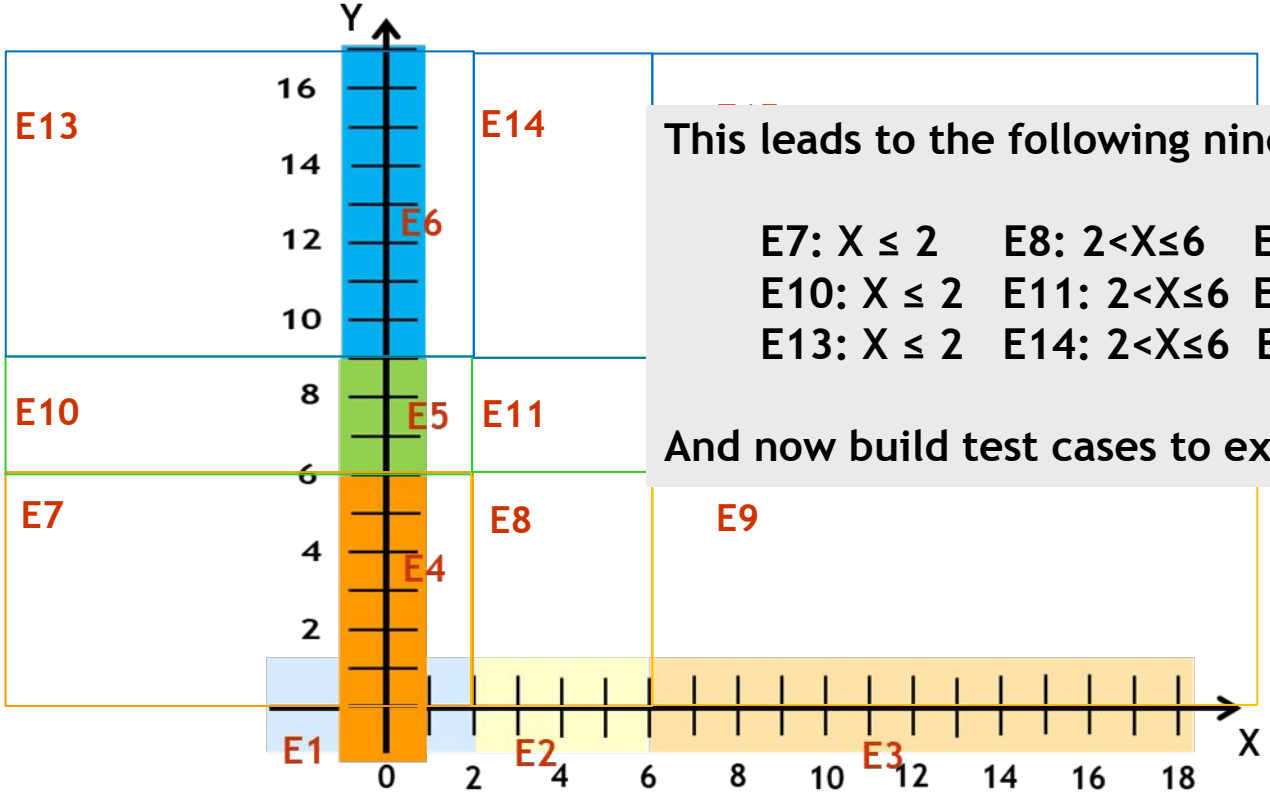
Equivalence class partitioning

- **Multidimensional partitioning**
  - Consider the input domain as the set product of the input variables.
  - One partition consisting of several equivalence classes.
  - We refer to this method as multidimensional equivalence partitioning or multidimensional partitioning.



Equivalence class partitioning

- Multidimensional partitioning
  - Consider the input domain as the set product of the input variables.
  - One partition consisting of several equivalence classes.
  - We refer to this method as multidimensional equivalence partitioning or multidimensional partitioning.



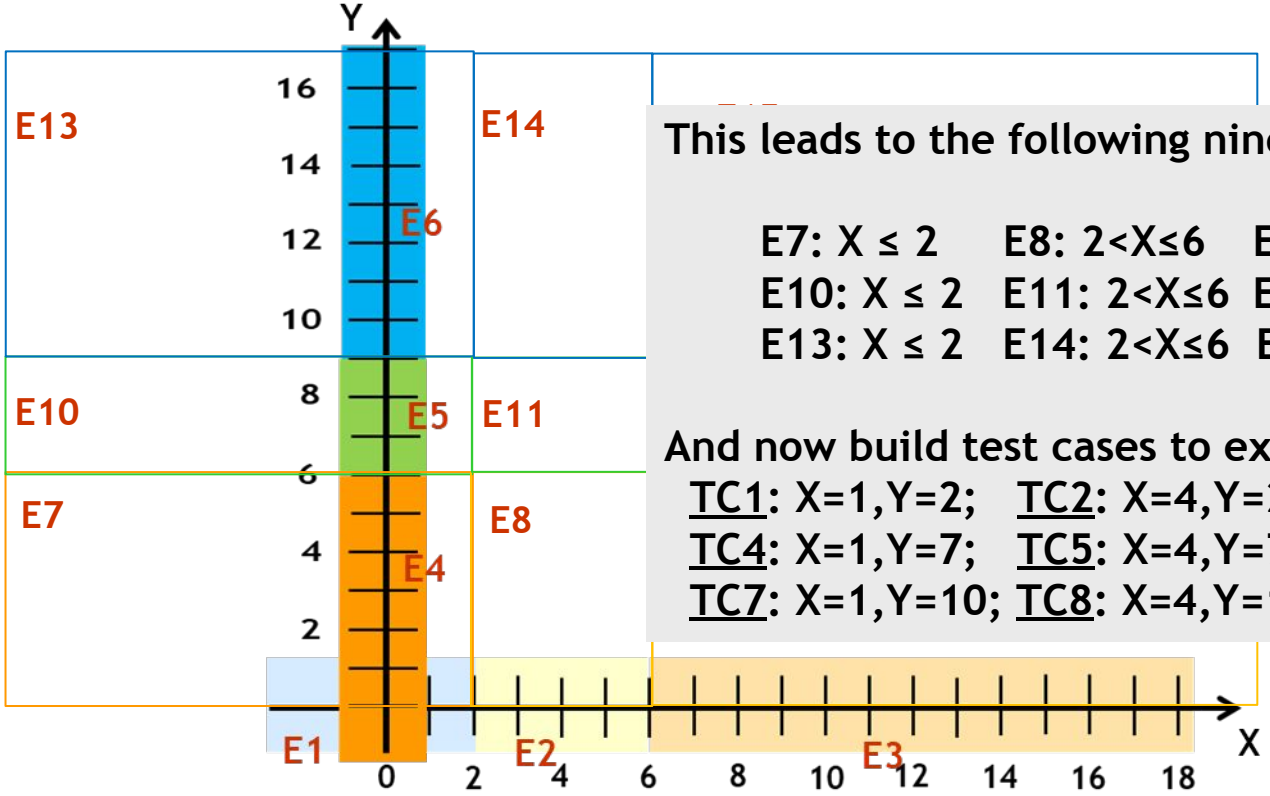
This leads to the following nine equivalence classes:

E7: X ≤ 2 E8: 2<X≤6 E9: X>6 (Y<6)
E10: X ≤ 2 E11: 2<X≤6 E12: X>6 (6≤Y≤9)
E13: X ≤ 2 E14: 2<X≤6 E15: X>6 (Y>9)

And now build test cases to exercise those 9 classes

Equivalence class partitioning

- Multidimensional partitioning
  - Consider the input domain as the set product of the input variables.
  - One partition consisting of several equivalence classes.
  - We refer to this method as multidimensional equivalence partitioning or multidimensional partitioning.



This leads to the following nine equivalence classes:

E7: X ≤ 2 E8: 2<X≤6 E9: X>6 (Y<6)
E10: X ≤ 2 E11: 2<X≤6 E12: X>6 (6≤Y≤9)
E13: X ≤ 2 E14: 2<X≤6 E15: X>6 (Y>9)

And now build test cases to exercise those 9 classes

TC1: X=1,Y=2; TC2: X=4,Y=2; TC3: X=8,Y=2
TC4: X=1,Y=7; TC5: X=4,Y=7; TC6: X=8,Y=7
TC7: X=1,Y=10; TC8: X=4,Y=10; TC9: X=8,Y=10

Pairwise testing

Pairwise testing: A black box test design technique in which test cases are designed to execute all possible discrete combinations of each pair of input parameters

- The number of pairwise test cases will be $O(mn)$, where m and n are the number of possible values for the two parameters with the largest sets of values

$$O(mn) = max(\Omega) \cdot max(\Omega \setminus max(\Omega))$$

- and Ω is the set with the number of possible values for every parameter

- Let's see an example

Pairwise testing

Consider 3 parameters and possible values:

Parameter Name	Value 1	Value 2	Value 3	Value 4
Enabled	True	False		
Type	1	2	3	
Category	A	B	C	D

Pairwise testing

Consider 3 parameters and possible values:

Parameter Name	Value 1	Value 2	Value 3	Value 4
Enabled	True	False		
Type	1	2	3	
Category	A	B	C	D

“Enabled” has 2 possible values
“Type” has 3 possible values
“Category” has 4 possible values

So,
$\Omega = \{2, 3, 4\}$
$m = \max(\Omega) = 4$
$n = \max(\Omega \backslash \max(\Omega)) = \max(2,3) = 3$

$O(mn) = 4 \times 3 = 12$

Pairwise testing

Consider 3 parameters and possible values:

Parameter Name	Value 1	Value 2	Value 3	Value 4
Enabled	True	False		
Type	1	2	3	
Category	A	B	C	D

"Enabled" has 2 possible values
"Type" has 3 possible values
"Category" has 4 possible values

So,
$\Omega = \{2, 3, 4\}$
$m = \max(\Omega) = 4$
$n = \max(\Omega \backslash \max(\Omega)) = \max(2,3) = 3$

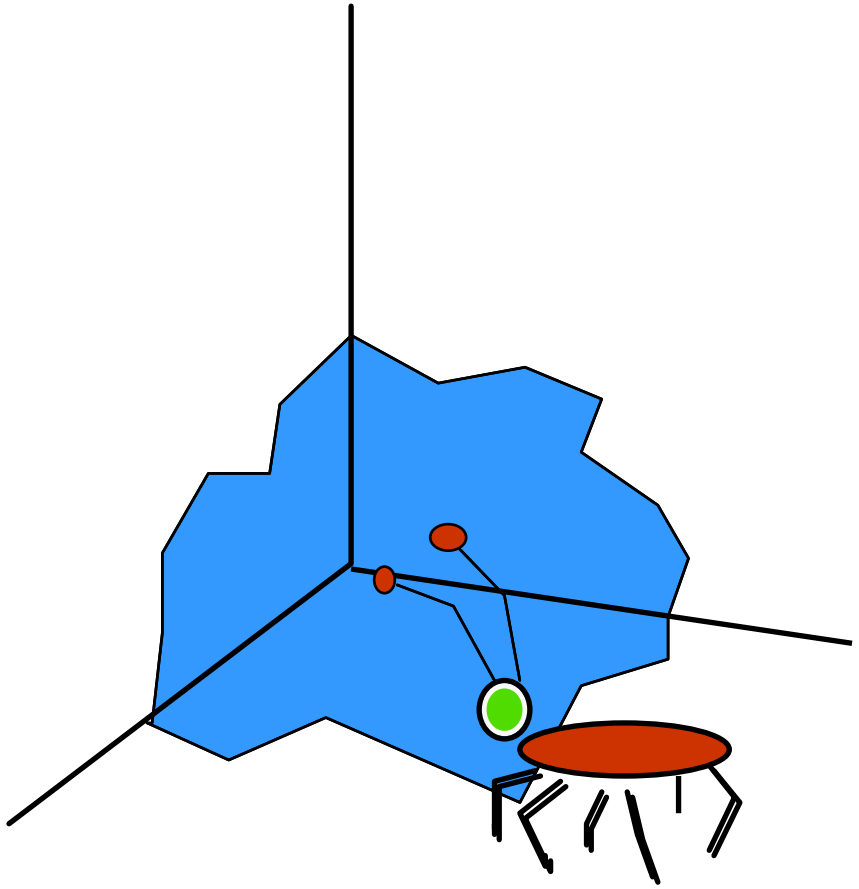
$O(mn) = 4 \times 3 = 12$

Enabled	Type	Category
True	3	A
True	1	D
False	1	C
False	2	D
True	2	C
False	2	A
False	1	A
False	3	B
True	2	B
True	3	D
False	3	C
True	1	B

Boundary value analysis

“Bugs lurk in corners and congregate at boundaries.”

[Boris Beizer, "Software testing techniques"]



Boundary value analysis

- Boundary testing is the process of testing between extreme ends or boundaries between partitions of the input values.

  - Off-by-one mistakes are a common cause for bugs in software systems. As developers, we have all made mistakes such as using a "greater than" operator (>) where it had to be a "greater than or equal to" operator (>=). Interestingly, programs with such a bug tend to work well for most of the provided inputs. They fail, however, when the input is "near the boundary of condition".

Boundary value analysis

Based on experience/heuristics:

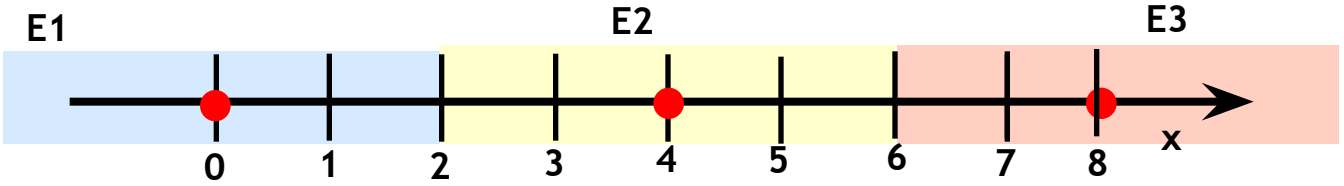
- Testing boundary conditions of equivalence classes is more effective, i.e., values directly on, above, and beneath edges of classes
  - If a system behaves correctly at boundary values, then it probably will work correctly at “middle” values
- Choose input boundary values as tests in input classes instead of, or additional to arbitrary values
- Choose also inputs that invoke output boundary values (values on the boundary of output classes)
- Example strategy as an extension of equivalence class partitioning:
  - Choose one (or more) arbitrary value(s) in each equivalence class
  - Choose values exactly on the lower and upper boundaries of equivalence classes
  - choose values immediately below and above each boundary (if applicable)

Boundary value analysis

- **On-point**: The value that is exactly on the boundary. This is the value we see in the condition itself.
- **Off-point**: The value that is closest to the boundary and that flips the condition's result. If the on-point makes the condition evaluate to true, the off-point makes it evaluate to false and vice versa.
  - Note that when dealing with equalities or inequalities (e.g., x == 6 or x != 6), there are two off-points; one in each direction.
- **In-points**: All values that make the condition evaluate to true.
- **Out-points**: All values that make the condition evaluate to false.

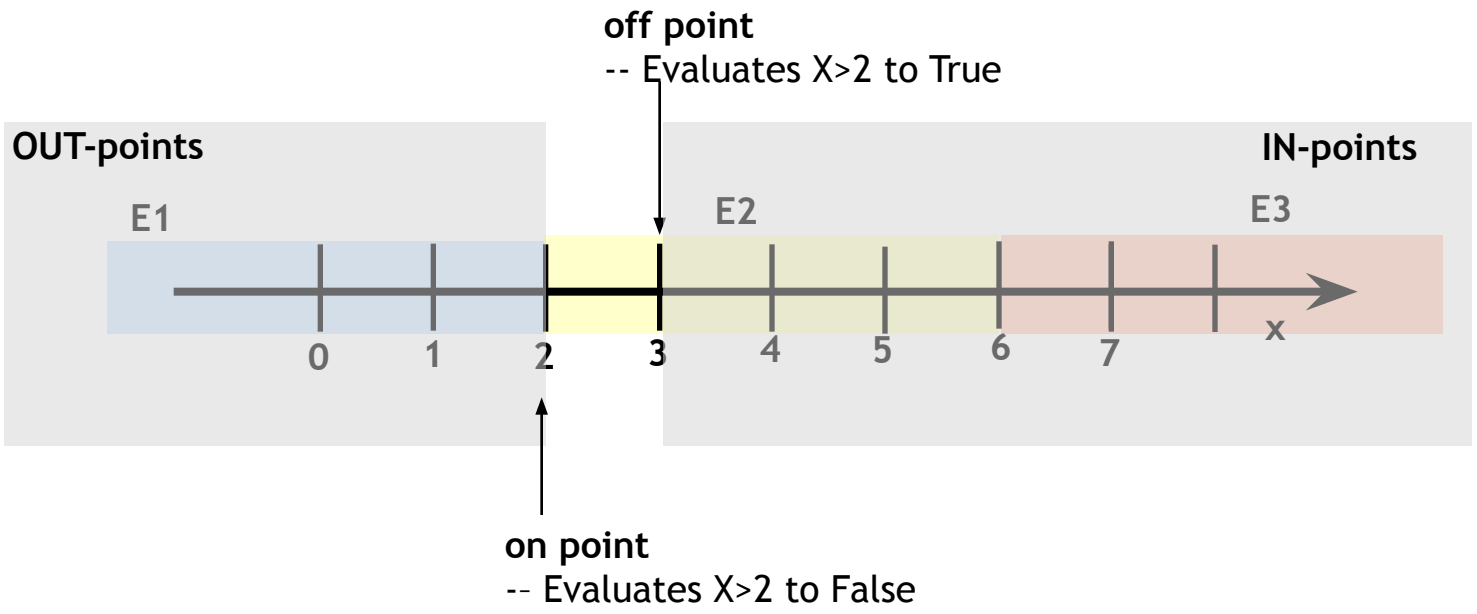
Boundary value analysis

- Consider again an application that
  - inputs one integer denoted by X
  - the only legal X values are in the range]2..6], i.e., X>2 and X<=6
- BVA (Boundary value analysis) includes ECP (equivalence class partitioning).
- So, 3 test cases for ECP (one per class), for example,
  - test 1: X=0, test 2: X=4, and test 3: X=8.



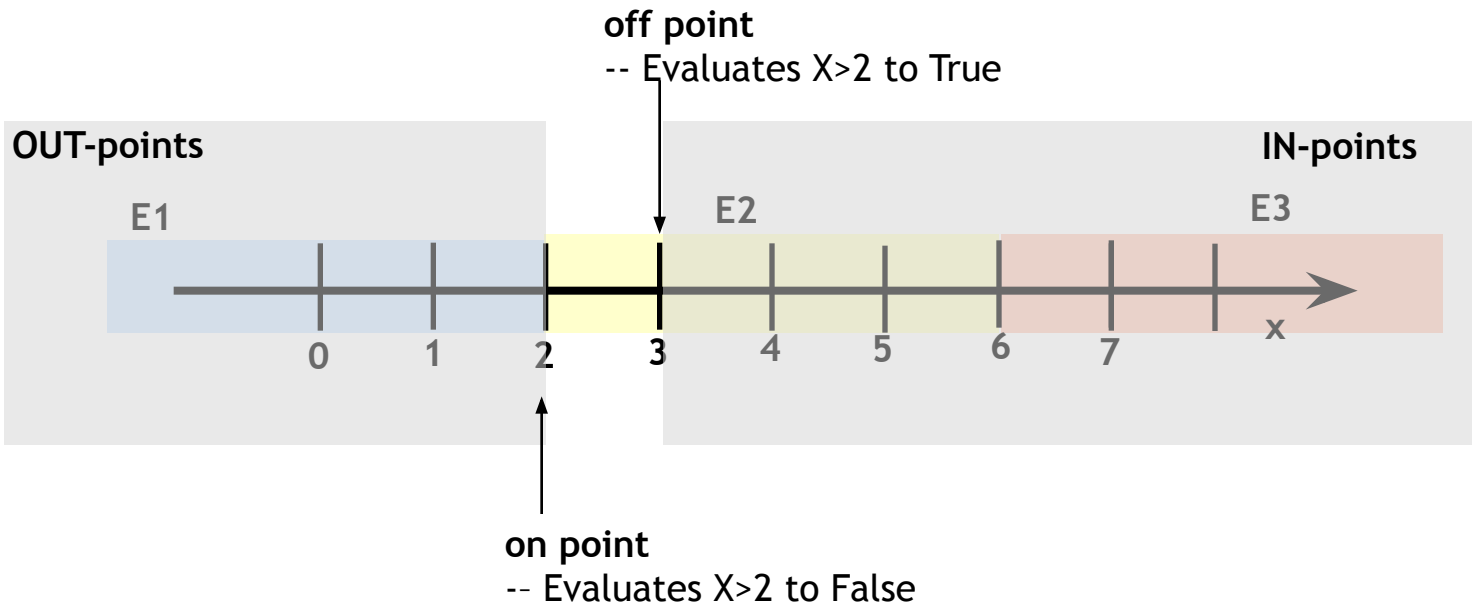
Boundary value analysis

- And additionally, devise tests to explore the boundaries
- Let us explore the boundaries (X>2 and X<=6)



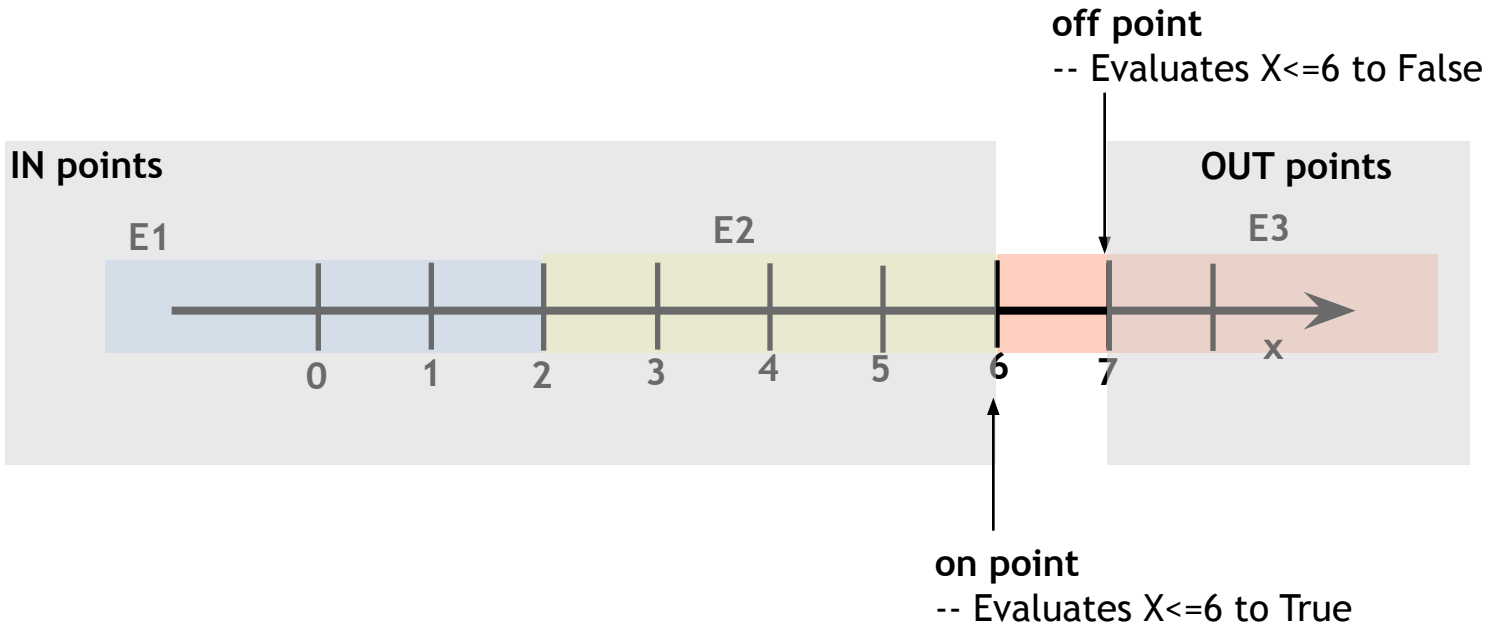
Boundary value analysis

- And additionally, devise tests to explore the boundaries
- Let us explore the boundaries (X>2 and X<=6).
- 2 test cases: test 4: X=2 (class E1); test 5: X=3 (class E2)



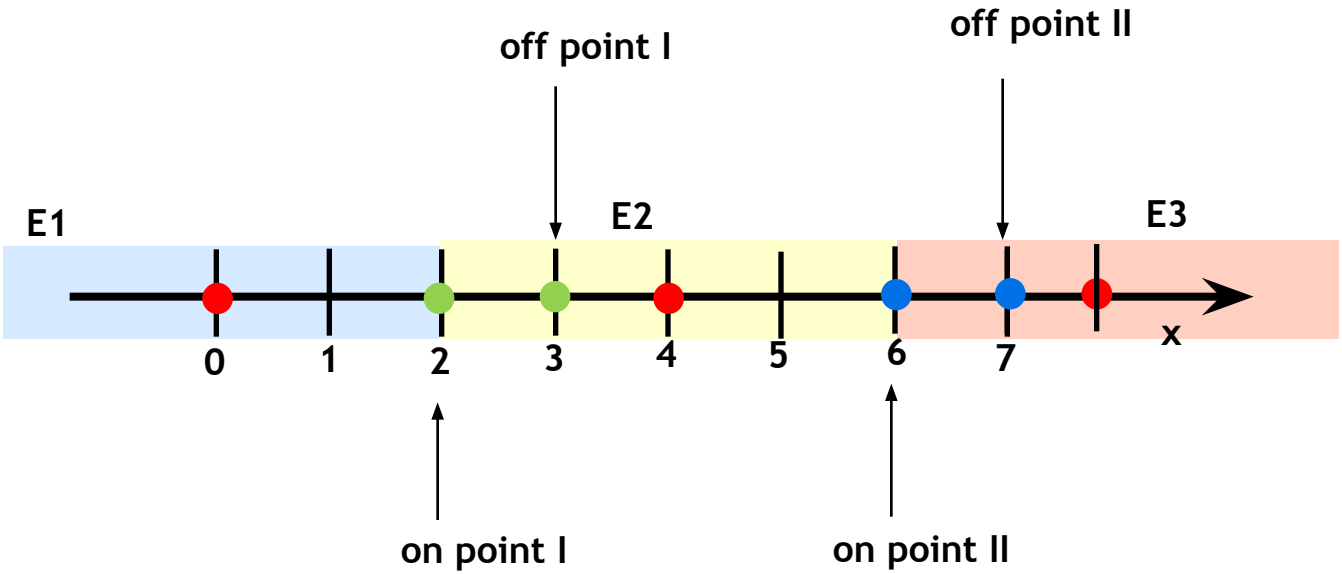
Boundary value analysis

- And additionally, devise tests to explore the boundaries
- Let us explore the boundaries (X>2 and X<=6).
- 2 test cases: test 6: X=6 (class E2); test 7: X=7 (class E3)



Boundary value analysis

- 3 test cases, one per partition (ECP)
  - test 1: X=0 (E1), test 2: X=4 (E2), and test 3: X=8 (E3)
- 2 test cases for boundary I (X>2)
  - test 4: X=2 (E1); test 5: X=3 (E2)
- 2 test cases for boundary II (X<=6)
  - test 6: X=6 (E2); test 7: X=7 (E3)



Cause-effect graphing

- Black-box technique to analyze combinations of input conditions
- Identify causes and effects in specification

  ↓ ↓

  inputs / initial state conditions

  outputs / final state conditions
- Make a Boolean Graph linking causes and effects
- Annotate impossible combinations of causes and effects
- Develop a decision table from a graph with each column a particular combination of inputs and outputs
- Transform each column into a test case

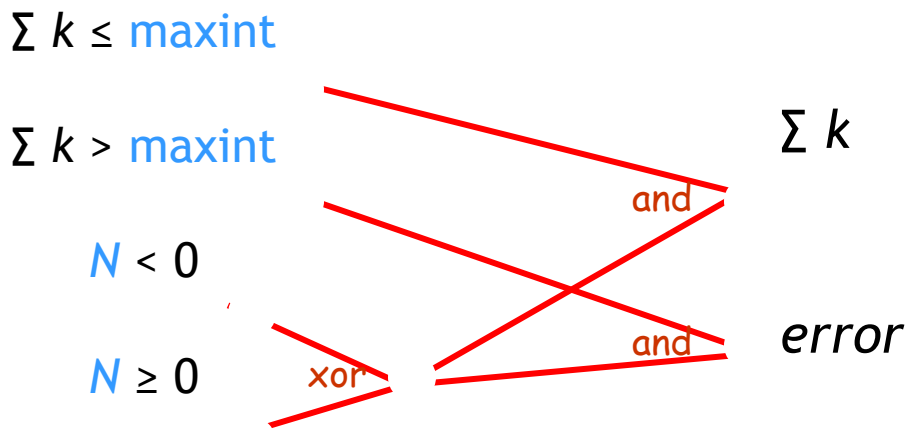
Cause-effect graphing (example)

- Consider a program that
  - computes the *sum* of the first *N* integers, as long as this *sum* is less than *maxint*.
  - Otherwise, an error should be reported.
  - If *N* is negative, then it takes the absolute value *N*.
- Formally:

  Given integer inputs *N* and *maxint* compute result :

$$\text{result} = \sum_{K=0}^{|N|} k \quad \text{if this} <= \text{maxint, error otherwise}$$

Cause-effect graphing



Decision table

("truth table")

Each entry in de decision table is a 0 or a 1 depending on whether or not the corresponding condition is false or true

causes	∑ *k* ≤ maxint	1	1	0	0
(inputs)	∑ *k* > maxint	0	0	1	1
	N < 0	1	0	1	0
	N ≥ 0	0	1	0	1
effects	∑ *k*	1	1	0	0
(outputs)	*error*	0	0	1	1

Cause-effect graphing

- Systematic method for generating test cases representing combinations of conditions
- Differently from eq. class partitioning, we define a test case for each possible combination of conditions
- Combinatorial explosion of the number of possible combinations
  - In the worst case, if *n* causes are related to an effect *e*, then the maximum number of combinations is 2^n.

Error guessing

- Just ‘guess’ where the errors are ……
- Intuition and experience of tester
- Ad hoc, not really a technique
- But can be quite effective
- Strategy:
  - Make a list of possible errors or error-prone situations (often related to boundary conditions)
  - Write test cases based on this list

Random testing

- Input values are randomly generated
- Do you know that a monkey using a piano keyboard could play a Vivaldi opera? Could the same monkey, using your application, discovery defects?
- Two kinds of tools
  - **Dumb monkeys** – low IQ; they can't recognize an error when they see one
  - **Smart monkeys** – generate inputs with some knowledge to reflect expected usage; get knowledge from state table or model of the AUT.
- Microsoft says that 10 to 20% of the bugs in Microsoft projects are found by these tools

Random testing

- Advantages
  - Good for finding system crashes
  - Particularly adequate for performance testing (it's not necessary to check the correctness of outputs)
  - No effort in generating test cases
  - Independent of updates
  - Increase confidence on the software when running several hours without finding errors
  - “Easy” to implement
- Disadvantages
  - Not good for finding other kinds of errors
  - Difficult to reproduce the errors (repeat test cases / sequence of inputs)
  - Unpredictable
  - May not cover special cases that are discovered by "manual" techniques

Black box testing: Which One ?

- Black box testing techniques :
  - Equivalence partitioning
  - Boundary value analysis
  - Cause-effect graphing
  - Error guessing
  - ……
- Which one to use ?

Black box testing: Which One ?

- Black box testing techniques :
  - Equivalence partitioning
  - Boundary value analysis
  - Cause-effect graphing
  - Error guessing
  - ……
- Which one to use ?
  - None of them is complete
  - All are based on some kind of heuristics
  - They are complementary

Black box testing: which one ?

- Always use a combination of techniques
  - When a formal specification is available try to use it
  - Identify valid and invalid input equivalence classes
  - Identify output equivalence classes
  - Apply boundary value analysis on valid equivalence classes
  - Guess about possible errors
  - Cause-effect graphing for linking inputs and outputs

HOMEWORK

Exercise 1

"A program reads three integer values. The three values are interpreted as representing the lengths of the sides of a triangle. The program prints a message that states whether the triangle is scalene (all lengths are different), isosceles (two lengths are equal), or equilateral (all lengths are equal)."

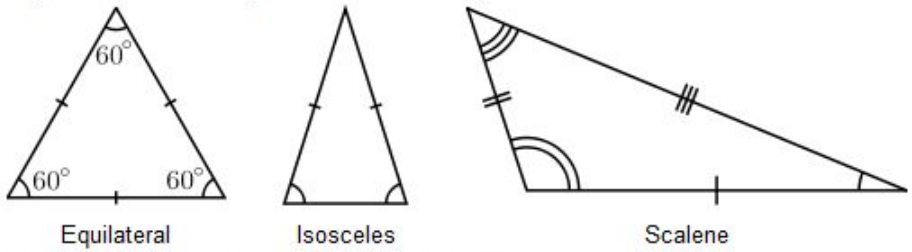
Write a set of test cases to test this program.

Inputs: A, B, C, integers

A>0, B>0, C>0

A < B+C, B < A+C, C < B+ A

Output: error, scalene, isosceles or equilateral



Exercise 2

Given inputs maxint and N compute result :

$$\text{result} = \sum_{K=0}^{|N|} k \quad \text{if this <= maxint, error otherwise}$$

Write a set of test cases to test this program

Exercise 3

Write a set of test cases to test this program

```
procedure Search (Key : ELEM ; T: ELEM_ARRAY;
      Found : out BOOLEAN; L: out ELEM_INDEX) ;

Pre-condition
      -- the array has at least one element
      T'FIRST <= T'LAST
Post-condition
      -- the element is found and is referenced by L
      ( Found and T (L) = Key)
or
      -- the element is not in the array
      ( not Found and
      not (exists i, T'FIRST >= i <= T'LAST, T (i) = Key ))
```

References

- Practical Software Testing, Ilene Burnstein, Springer-Verlag, 2003
- Software Testing, Ron Patton, SAMS, 2001
- The Art of Software Testing, Glenford J. Myers, Wiley & Sons, 1979 (Chapter 4 - Test Case Design)
  - Classical
- Software testing techniques, Boris Beizer, Van Nostrand Reinhold, 2nd Ed, 1990
  - Bible
- Foundations of Software Testing, 1st edition, by Aditya P. Mathur, Pearson Education, 2008
- Testing Computer Software, 2nd Edition, Cem Kaner, Jack Falk, Hung Nguyen, John Wiley & Sons, 1999
  - Practical, black box only
- Software Engineering, Ian Sommerville, 6th Edition, Addison-Wesley, 2000
- Test Driven Development, Kent Beck, Addison Wesley, 2002
- http://www.swebok.org/
  - Guide to the Software Engineering Body of Knowledge (SWEBOK), IEEE Computer Society
- http://www.mccabe.com/iq_research_metrics.htm
- What Is a Good Test Case?, Cem Kaner, Florida Institute of Tecnology, 2003
- Software Unit Test Coverage and Adequacy, Hong Zhu et al, ACM Computing Surveys, December 1997